

Airline Backend System

Objective: We need to build a backend system that can support different features for an airline company. Our end user is going to be someone who wants to book flights and query about flights. So, we need a robust system to actually help them give the best possible experience. This doc is solely going to focus on the backend part of the system. We want to prepare the whole backend keeping in mind that the codebase should be as maintainable as possible.

Requirements:

- A user should be able to search for flights from one place to another.
 - Users should be able to mention the source and destination details.
 - Users should be able to select the date of the journey.
 - [V2] User should be able to search for return flights and multi-city flights.
 - Users should be able to select the class of the flights. [Non-mandatory, default value]
 - Users should be able to select the number of seats to book.
 - Based on the above data, the system should list down the available flights with details such as selecting from multiple airports, duration of the flight, stops or non-stop, carbon emissions [if, not mandatory], price (as per round trip or single trip), flight reviews.
 - Users can sort/ filter the results based on the parameters such as cheapest first, or least time taking, arrival time, etc.
 - System needs to support the pagination feature which means the system only needs to display a chunk at one time.
- A user should be able to book a flight considering the user is registered onto the platform.
 - Users should be able to cancel a booking, and then based on some internal criteria, the system should initiate a refund.
 - Users should be able to request and book excess luggage for every flight.
 - A user should be able to book more than one ticket/ seat [with the add passenger details option] with one login id.
- To confirm the booking, the user has to make a payment [We will make a dummy mimic].
- Tracking flight prices should be a feature, the user should be notified about any price drops or any delays. [Users need not book the flight, but they can opt to track the flight details].
- A user should be able to list their previous flight booking details as well as the upcoming booked ones.
- Users should be able to download boarding pass if they've done online check-in.
- Online checkin mechanism should be supported.
- Notifications via email for completing online check-in before three hours of departure.
- Notify all the users of any flight delays or changes.
- Users should be able to review the flight journey iff they've booked the flight. [After the journey is completed].
 - Review mechanism should involve star ratings along with a comment.
 - While listing any flight, the system must also display the flight reviews.

- Users should be able to authenticate in the system using their email and password.
- [V2] Support ticketing, where users can raise their queries.
- [V2] Preparing seat selection.
- Listing FAQs which will be static data.
- Coupons for offers and discounts.

Non-Functional Requirements:

- We are going to have more flight searches than the number of bookings.
- The system needs to be accurate in terms of booking.
- Expect that we will be having approximately 1,00,000 total users; 5,00,000 bookings might come in one quarter.
- So we may expect around 10,000 flight bookings per day.
- System should be capable of scaling up to at least 3 times the current estimated traffic.
- The system should handle real-time updates to the flight prices, before user makes the final booking.
- Concurrency should be handled, using RDBMS should be a good solution.

Capacity Estimation:

- Storage estimate:
 - For upcoming 5 years (~80,00,000 bookings, ~2,00,000 users).
 - Considering all the user records and booking records takes 10 MB of data then overall 10TB of memory should be fine for our initial pilot run.
- Traffic estimate:
 - If we consider 30:1 as the search:booking ratio, then at max we expect approximately 1,50,000 search queries a day; around 2 queries/sec

Search and Flights Service:

- Create Flights
- Delete Flights
- Update Flights
- Search for Flights
 - Based on multiple filters
 - Pagination

Database Design

1. Entities:

- **Airplane**
- **Flight**
- **Airport**
- **City**

2. Attributes:

- **Airplane Table:**
 - ID (Primary Key)
 - Model_number
 - Capacity
 - Created_at
 - Updated_at
- **Flights Table:**
 - Id (Primary Key)
 - Src_airport_id (Foreign Key)
 - Dest_airport_id (Foreign Key)
 - Departure_date
 - Arrival_date
 - Departure_time
 - Arrival_time
- **City Table:**
 - Id (Primary Key)
 - Name
- **Airport Table:**
 - Id (Primary Key)
 - Name
 - City_id (Foreign Key)
 - Address

3. Relationships:

- One airplane can be used for many flights (One-to-Many).
- One city can have many airports (One-to-Many).
- One airport can have many flights, but one flight can only belong to one airport (One-to-Many).

